Politecnico di Torino

Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2025/2026

Software Design and Architecture

Docenti:
Antonio Vetro'
Marco Torchiano

Studente:
Cristiano Berardo

Indice

Capitolo 1

Introduction to Software Design

1.1 Complexity

Complexity is anything related to the structure of a software system that makes it hard to understand and modify the system.

$$C = \sum c_p t_p$$

Where C is determined by all software parts p, c_p^p is the complexity of part p, and t_p is the fraction of time spent on part p.

1.1.1 Syntax complexity

- Change amplification: change require operating in different places
- Cognitive load: how mutch information to learn to understand and perform a task. Some times is better to write longer code to reduce the complexity and so the cognitive load.
- Unknown unknowns: there are partes of the system that we don't know where they are or even that they exist.

1.1.2 Cause of complexity

Dependency: When we need to understand or modify a part of the code, we need to understand or modify other parts of the code. Try to reduce the number of dependencies and the complexity of the dependencies.

Obscuiry: alla the important information shoul be clearly visible.

1.1.3 Strategical vs. Tactical Programming

- **Tactical programming**: focus on having code that works as fast as possible. "Move fast and break things".
- **Strategical programming**: focus on long term maintainability and scalability. "Working code is not enough".

1.2 Modules

To improve and eliminate dependency and complexity, we can use modules. A module is a part of the code that has a well defined interface and a well defined implementation.

- **Interface**: what we need to know to use the module. Formal part, possible checked by the compiler. Represents a cost.

- **Implementation**: how the module works. Ideall much more complex than the interface. Represents a benefit.

1.2.1 Abstraction

What we tipically do is to perform an abstraction. Abstraction means selective removal of information. Is a simplified view of the something that omits "unimportant" information. Omitting information can lead to eads to unknowns and keeping unimportant information can leads to cognitive overload.

Deep modules and shallow interfaces

Deep modules: modules with a simple interface and a complex implementation. They are good because they reduce the cognitive load and the change amplification. They are bad because they can lead to unknown unknowns.
Shallow interfaces: modules with a complex interface and a simple implementation. They are good because they reduce the unknown unknowns. They are bad because they can lead to cognitive overload and change amplification.

1.2.2 Information Hiding

Information hiding is the principle of hiding the implementation details of a module from the users of the module. Example sort function: we want to sort a list of numbers, we don't care about how the sorting algorithm used.

1.2.3 Information Leakage

Is the opposite of information hiding. Occurs when the implementation details of a module is reflected in multiple modules with consequences of creating dependencies.

1.2.4 Information Hiding Red Flags

- **temporal decomposition**: execution order is reflected in the code structure. Operations that happen in different times are in different classes.
- **overexposure**: API forces knowledge about rarely used features to use common ones. E.g. `BufferedReader r = new BufferedReader(new InputStreamReader(System.in));`
- **Too many classes**: code is divide into too many shallow classes.

1.2.5 Method separation

Better together or Better apart?

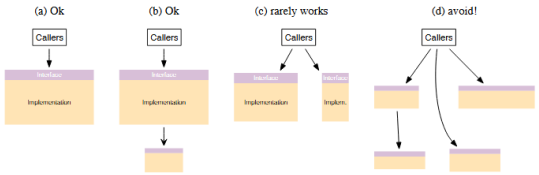
- **Bring together**: if information is shared, if we simplify the interface, if we eliminate duplication.
- **Separate**: if the code is general purpose or specific purpose.

Method separation red flags

If we have a lot of repeded code and/or we mixed specific and general code, means we have put together code that should be separated.

1.2.6 Do one simple thing and complexity

Each method should do one thing and do it completely



1.3 Obscurity

Obscurity is the principle of making the important information clearly visible and the unimportant information hidden. Obscurity is a great source of complexity. Obscure code is hard to Understand and creates more Bugs.

The solution to obscurity is writing obvious code, this means with a quick glance we can understand what the code does. We can achieve this by using abstraction layers.

1.3.1 Abstraction layers

Software systems are composed in layers. Higher layers use the facilities provided by lower layers.

Abstraction simplifies understanding and reduces obscurity.

In a well-designed system, each layer provides a different abstraction from the layers above and below it.

Abstraction Layers Red Flags

Some red flags that indicate that we have a problem with abstraction layers are:

- **Pass-Through methods**: is a method that does nothing but call another method.
- **Pass-Through variables**: is a variable that is passed through over and over down a long chain of method calls.

1.3.2 Writing Comments

Another method to reduce obscurity is writing comments. If users need to read a method code to use it, there is no abstraction.

Comments should describe things that are not obvious in the code:

- **Pick conventions**: write comments like other persons in the project
- **Don't repeat code**: avoid redundant comments that simply restate what the code already makes clear
- **Higher level comments enhance intuition**
- **Delayed comments are bad comments**
- **Use comments as design tool**

1.3.3 Naming

Toghether with comments, naming is a great tool to reduce obscurity. Bad names cause bugs. Name should create a image in mind of the reader about the nature of the thing being named. Names should be precise and consistent.

1.4 Errors and Anomalies

Exceptions add complexity because they create dependencies between the code and if they are easy to throw they are hard to handle.

A piece of code might throw exceptions in different ways:

- Caller may provide bad arguments or configuration
- Invoked method may not able to complete the request
- In distributed systems network packets may be lost or delay or peers may communicate in unxpected ways
- The code may detect bugs or internal inconsistencies or situations that are not prepared to handle

1.4.1 Define Errors out of Existence

What are the solutions to errors? Define errors out of existence. This means, not ignore the error but to redefine the behavior of a method so that no error is required.

E.g. `String.substring(int beginIndex, int endIndex)` it throws different exceptions Some in only specific cases. This is completly avoidable and unnecessary.

1.4.2 Limit exceptions

There are some methods to limit the number of exceptions that a method can throw:

- Handle exceptions at the point where they are thrown, if possible
- Exception aggregation means handle different exceptions with same handler
- Just crash
- Mask exceptions avoiding unexpected exceptions

1.5 Dependency Dimensions

1. Structural (Code-Level) Dependencies
2. Runtime / Distribution Dependencies
3. Data-Level Dependencies
4. Behavioral Dependencies

1.5.1 Structural (Code-Level) Dependencies

- **Implementation Dependency**: some piece of code depends on a concrete class instead of an abstraction
- **Construction Dependency**: you depend on how a collaboration, another class, is created.
- **Compile-Time Dependency**: the module knows something at compile time

1.5.2 Runtime / Distribution Dependencies

Often linked to distributed architectures.

- **Spatial Dependency**: A component depends on the location of another component. E.g. hard-coded URLs
- **Temporal Dependency**: A component depends on another being available at the same time. E.g. remote method calls or servers
- **Synchronization Dependency**: A component depends on another to complete work before continuing. E.g. synchronous request/response

1.5.3 Data-Level Dependencies

- **Data Dependency**: One component depends on another's internal data model. E.g. shared database
- **Schema Dependency**: Multiple services depend on the same database schema or representation structure. E.g. DB schema, XML/Json schema

1.5.4 Behavioral Dependencies

- **Control Flow Dependency**: One component dictates another's execution path. E.g. Centralized process managers
- **Transactional Dependency**: E.g. Components rely on atomic distributed consistency.
- **Timing Assumption Dependency**: Assumptions about response time or ordering. E.g. "Service must respond in <50ms" or Event ordering assumptions

Capitolo 2

Design Patterns

Initially proposed by Christopher Alexander in the context of architecture.

A reusable solution
to a known problem
in a well defined context

The context is a *design* situation giving rise to a (design) problem.
The problem is a set of forces repeatedly arising in the context. [1]
The solution is a proven resolution of the problem. Configuration to balance forces. Structure with components and relationships and run-time behaviour.

Example of Social Pattern: You are in queue to the supermarket gastronomic section. The problem is hard to spot who arrived first. The solution is to use a ticket machine.

2.1 Types of Software Patterns

- **Architectural patterns** - high level structures of software systems. E.g., layered architecture, client-server, microservices.
- **Design patterns** - solutions to common design problems. E.g., Singleton, Observer, Factory Method.
- **Idioms** - language-specific patterns. E.g., RAII in C++, list comprehensions in Python.

2.1.1 Architectural pattern

Normally the architectural pattern works with big components and their interactions and is needed to describe the responsibilities of these components. Expresses a fundamental structural organization schema for software systems and defines the rules and guidelines for organizing the relationships between the components.

Example of architectural pattern: Several programs that are used in sequence read from input and write sequentially to output.
The problem is there are a lot of intermediate files used for communication between programs.
The solution is to adopt pipes and filters architecture feeding with the result of the previous one.

2.1.2 Design patterns

Provides a scheme for refining components of a software system or their relationships and describes a commonly recurring structure of communicating components.

[1] Any relevant aspect of the problem E.g., requirements, constraints, desirable properties

Example of design pattern: A class library providing few functionalities contains a lot of classes.
The problem is the user is exposed to the internal complexity of the library.
The solution is to create a new façade class that interacts with the user and hide all the details.

2.1.3 Idioms

It is a low-level pattern specific to a programming language. Describes how to implement particular aspects of components or the relationships between them. Leverages the features of a programming language.

Example of idioms: An attribute is constant and should be globally available to many classes.
The problem is opening access would allow unauthorized modifications and the attribute is repeated in every object.
The solution is to make public static final in Java language.

2.2 Pattern language

Pattern do not exist in isolation in fact can happen that two or more patterns are applied together, used to implement part of another pattern and introduce a problem solved by another
They are called Pattern Languages or pattern systems.

Collection of patterns together with guidelines for:

- Implementation
- Combination
- Practical use

Should:

- Count enough patterns
- Describe patterns uniformly
- Present relationships

2.3 Idioms

X-ability

Clonability, serializability, comparability, etc. some specific behaviour dealing with many heterogeneous classes. The solution is to derive from a class or interface that defines the required methods.

Example: In Java all methods extends the root class *Object* and provides the methods *toString()*, *equals()*, *hashCode()*, etc. that can be overridden to provide specific behaviour.

2.3.1 Strong typing vs. Duck Typing

"If it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck."
Strong typed language: You must define interfaces and classes to be able to call a specific method. Compile time type checking.
Dynamic/duck typed language: You just add a method to a class and you can call it. No compile time type checking.

2.3.2 Property

A property, more general than an attribute, is a named characteristic of an object typically accessed through well-defined operations called accessors, getters, and setters forming the public interface of the object and it is not necessarily an attribute of the object (e.g. the age can be derived from the date of birth).

2.4 Design Patterns

Description of communicating objects and classes that are customized to solve a general design problem in a particular context.

A design pattern names, abstracts, and identifies the key aspects of a common design structure that make it useful for creating a reusable object- oriented design.

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	1	1	2
	Object	4	6	10

2.4.1 Pattern selection

How to use patterns? First of all you need to know the problem you are trying to solve, knowing the patterns and then select the one that best fits your needs.

2.4.2 Creational patterns

- Class

  Factory Method

- Object

  Abstract Factory

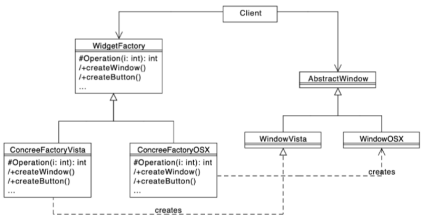
  Builder

  Prototype

  Singleton

Abstract Factory

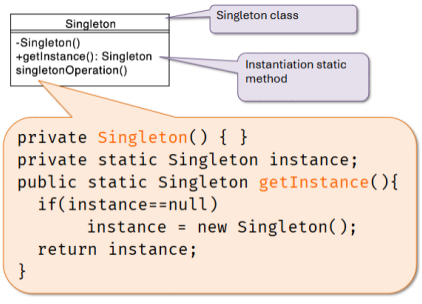
A family of related classes can have different implementation details.
The client should not know anything about which variant they are using / creating.



Example: GUI where you have different components (buttons, text fields, etc.) and the same GUI can be applied to different platforms such as Windows, MacOS, Linux, etc.

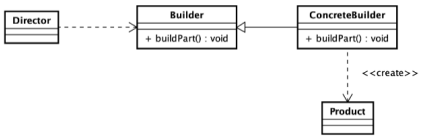
Singleton

A class represents a concept that requires a single instance.
The client should be able to access, in approper way, the unique instance.



Note: If we talk about of an object designer pattern, in python you can just create a global variable and it will be a singleton.

Builder



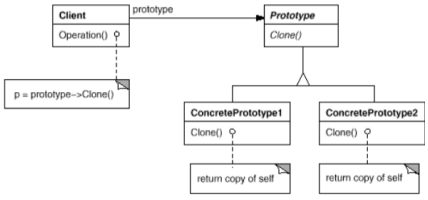
An object of a complex class has to be created.
The creation entails complex interaction with the object and could be different variations of the object.

Example: $10.40\,kg \cdot m^2 \cdot s^{-3}$

```
//Usual non-fluent builder
Measure power = new Measure(10.4);
power.addUnit("kg", 1);
power.addUnit("m", 2);
power.addUnit("s", -3);
power.setPrecision(2);
```

```
//Fluent builder
Measure power = new Measure.Builder(10.4)
    .is("kg")
    .by("m")
    .squared()
    .by("s")
    .to(-3)
    .withPrecision(2)
    .build();
```

Prototype



Several object can be created out of a palette of similar classes.
The class hierarchy can become quite complex and classes in hierarchy are differentiated by minor parameters (that could be defined through constructor arguments).
A solution could be start with a prototype classe and clone it to create new objects.

Example: A graphical editor where you have very similar shapes (e.g. square with different radius, colors, etc.).

2.4.3 Structural patterns

How organize classes and objects to compose larger structures.

- Class

  Adapter

- Object

  Bridge

  Composite

  Decorator

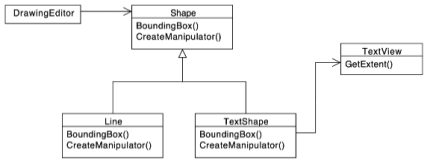
  Facade

  Flyweight

  Proxy

Adapter

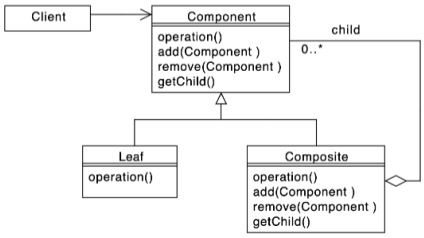
A class provides the features required by another class but its interface is not the one expected by the client.
The integration of the provider class should be possible without modifying it. The source code could be not available or is already used as it is somewhere else.



Example: In Java, keyboard events are handled by KeyListener. Listener class need to implement the interface and implement all the methods. If you are interested only in one of the methods, you can use the KeyAdapter class that is empty and you can decide which method to override, and the Listener extends KeyAdapter instead of KeyListener.

Composite

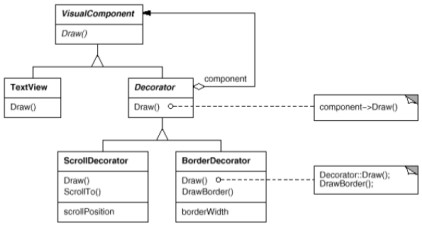
You need to represent part-whole hierarchies of objects.
Clients can be very complex and they require to adapt the difference between all the elements in the hierarchy.



Example: In Java UI components are organized in a tree structure. You can have a panel that contains other panels and buttons, etc.

Decorator

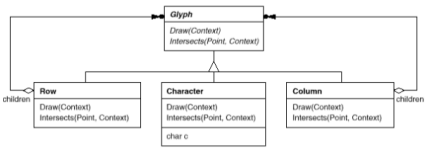
You have some objects of classes that have specific behaviour and you want to add the same behaviour plus someting else dynamically.
The solution can be extend the class but if that behaviour is something you would to apply to many classes, you need to extend all of leaf classes. Using a decorator you can modyfy the behaviour without creating an explosion of subclasses for every combination of features



Example: In Java we have JScrollPane can be used to provide a scroll pane to any component

Flyweight

You need need to create a very large number of similar objects, and memory usage and performance become concerns.



Example: In Java we can pool different objects togheter. The Integer class in Java.

Proxy

You need to controll access to an object.

We need to provide a placeholder that is able to mediate between the client needs and all the requirements you want to controll before accessing the final object without changing interface or class.

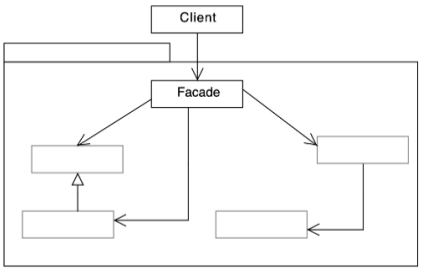
- **Remote proxy**: provides a local representative for an object in a different address space.
- **Virtual proxy**: creates expensive objects on demand. E.g. slow loading resource - images in a web page.

- **Protection proxy**: controls access to the original object. E.g. when objects should have different access rights.

- **Smart reference**: is a replacement for a bare pointer that performs additional actions when an object is accessed. E.g. counting the number of references, checking permissions, etc.

Facade

Hide the details and the complexity of a general class by providing a simpler interface. The client should not know anything about the subsystem.



2.4.4 Behavioral patterns

Behavioral patterns allow to implement specific algorithms using objects and classes to provide control and flexibility.

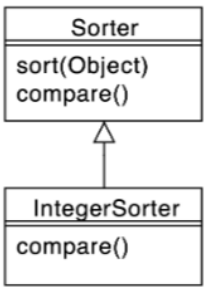
- Class
  - Interpreter
  - Template Method
- Object
  - Chain of Responsibility
  - Command
  - Iterator
  - Mediator
  - Memento
  - Observer
  - State
  - Strategy
  - Visitor

The basic mechanism of behavioral patterns is enapsultion variation. Instead of having a lot of if-else you have a structure at object level that allows you to define different alternatives.

2.4.5 Template Method

An algorithm/behavior has a stable core and several variation at given points. You have to implement/maintain several almost identical pieces of code.

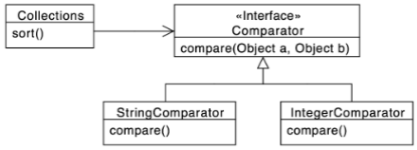
Example: The compare in Java. You have to implement the compare method that solves integer or string comparison.



2.4.6 Strategy Pattern

Many classes or algorithm has a stable core and several behavioral variations, several different implementations are needed. Every variation of an algorithm can be embedded in an object and passed as a parameter to the algorithm.

Example: Comparator in Java.
Consequences:



+ Avoid conditional statements

+ Algorithms may be organized in families

+ Choice of implementations

+ Run-time binding

- Clients must be aware of different strategies

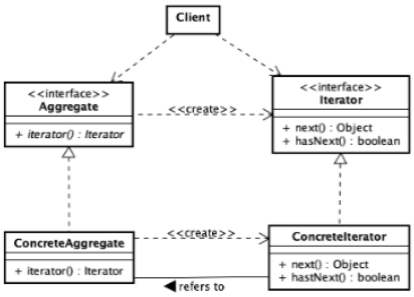
- Communication overhead

- Increased number of objects

2.4.7 Iterator pattern

You have a collection of objects and you want to iterate on it allowing multiple concurrent iterators.

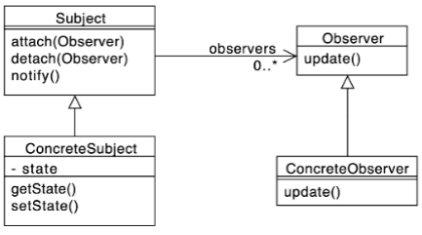
Example: In Java there is the Iterable interface and Iterator.



2.4.8 Observer pattern

The change in one object may influence one or more other objects. This can cause high coupling between objects and the number and type of objects to be notified may not be known in advance.

Example: In Java there is the Observable interface and Observer interface. This is very common in UI programming.
Consequences:

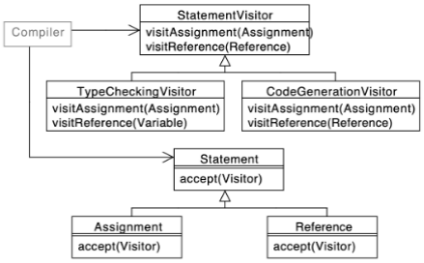


+ Abstract coupling between Subject and Observer

+ Support for broadcast communication

- Unanticipated updates

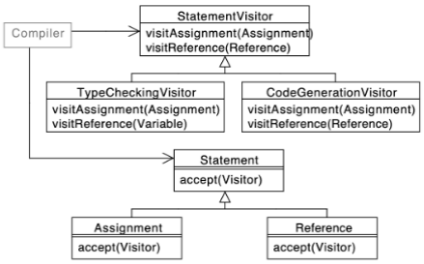
2.4.9 Visitor



You have an object structure that contains many classes with different interfaces and you want to perform several different and unrelated operations on these objects.
The operations depend on the concrete classes and you want to avoid putting many different methods in each of these classes.

2.5 Inheritance vs. composition

Reuse can be achieved via:



- **Inheritance**: a new class is created by inheriting from an existing class and adding new features or modifying existing ones. Clients can invoke directly inherited methods.
- **Composition**: The class has a link or instance to a class you want to reuse and just forwards the calls to it.

Note: The Observable pattern is deprecated. The PropertyChangeSupport is more flexible.

2.6 Abstract Syntax Tree

The AST is a tree that represent a syntax of formal language. We can use the composite pattern to representationt he arithmetic expressions.

And using the visitor pattern or the interpreter pattern to evaluate the expression.

2.7 Model View Controller

Capitolo 3

What is software architecture ?

The architecture of a software system is the shape given to that system by those who build it. The form of that shape is in the division of that system into components, the arrangement of those components, and the ways in which those components communicate with each other. The purpose of that shape is to facilitate the development, deployment, operation, and maintenance of the software system contained within it.

3.1 Goals of an architecture

The software must change to adapt to software regulation, user feedback.

We have some pattern that make methods and classes easy to be changed, but now we want something behond the methods.

The goals of an architecture are:

- keep the system valuable for its users over time
- minimize the lifetime cost of the system
- maximize programmer productivity

This can be achieved by making the system:

- easy to understand
- easy to develop
- easy to maintain
- easy to deploy
- easy to change

In Java we have the package, that is a module, and is a logical grouping of classes.

Design Patterns: helps to build methods and classes inside the module in a way that they are easy to be changed.

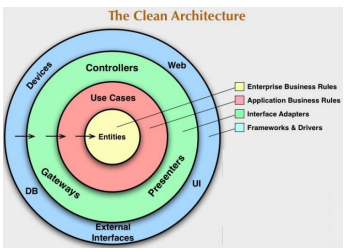
Design Principles: define the modules can collaborate together and how they are shaped.

3.2 The clear architectures

The **Outer circles**: mechanisms, they change often.

The **Inner circles**: policies, subject to less frequent changes.

Source code dependencies point only inward, toward higher-level policies (Dependency Rule)

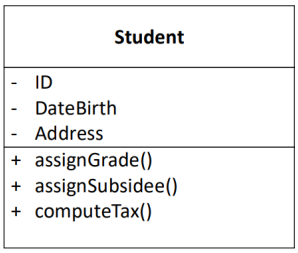


3.2.1 Entities

An Entity is an object within a software system that contains a set of critical business rules operating on Critical Business Data.

The interface of the Entity consists of the functions that implement the Critical Business Rules to operate on the Critical Business Data. They contain the most general and highlevel rules and neither they are often changed nor they are usually affected by lower-level changes.

Example: The system is the university portal and an important entity is the student, that has some critical business rules like "the student can enroll to the master degree only if they have completed the required prerequisites and then assign new ID".
Is the most stable part of the system and we want to preserve as much as possible from changes occurring to the other elements.



3.3 Use cases

To applay the Businness Roles to the entities we need to use the use cases.

A use case is a description of the software behavior and interactions (with people or other software) in a specific case.

It specifies:

- the input to be provided by the user (or other systems)
- the output to be returned
- The processing steps involved.

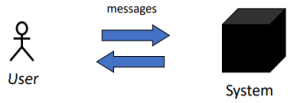
to determine the flow of data to and from the entities.
Software at this level change whenever a change to any element of a use case is required

Use case in pratice

It represents a **functional requirement**.

- It makes clear what you expect from a system ("what?")
- It hides the behaviour of the system ("how?")
- It is a sequence of actions (with variants) that produce a result observable by an actor

The system is seen as a black-box. Use cases express the interaction (exchange of messages) between the entities that interact with the system itself (called actors).



Example - Specify annual tax for a student Input: student ID
Output: annual tax amount

Main scenario:

1. Validate ID provided by user
2. Retrieve economic level
3. Compute due past taxes
4. Compute new taxes
5. Get average grades for the last academic year
6. If average grade > 26, apply 20% discount
7. Ask operator for confirmation: if yes change student else reset

3.4 Interface adapters

Their goal is to shape data in a way that it is convenient to higher levels or other systems or artifacts interacting with:

The same is done in the opposite direction. Examples of code in this layer: Presenters for GUI, Controller for DB

Code inward to this level know anything about details (e.g., SQL)

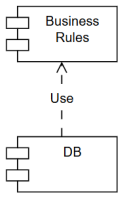
3.5 Frameworks and drivers

In order to achieve the separation we have seen so far, we need to separate low level details from the high-level details. Layers allow independent development, deployment, operation, and maintenance.

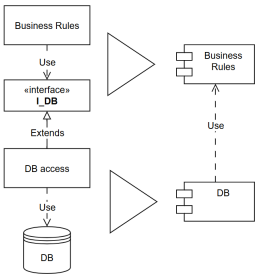
The details are the frameworks, the database, the drivers, the web server, etc., and usually we do not write mutch code here.

3.6 Separation of policy from details

Decoupling of policy from the details and defer decisions on details for as long as possible (MongoDB or SQL, etc.)
DB component knows about use case rules component, but NOT viceversa.
Low level components know everything of the higher-level components, but NOT viceversa



The Database component contains the code that translates the calls made by the BusinessRules into the query language of the database. This translation code knows about the BusinessRules.
Business rules can be adapted to any kind of DB.



Capitolo 4

From design to architecture: Design Principles

Design principles means follow a clean architecture and a clean architecture means that the architecture can easily evolve, to be changed.

4.1 The SOLID design principles

The SOLID principles suggest software architects on how to arrange functions and data into groupings (e.g., classes), and how these groupings are arranged.

The goal of the SOLID principles is to create mid-level software structures (modules) that:

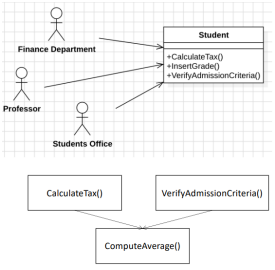
- Are easy to understand
- Allow changes in the most "comfortable" way
- Are the basis of reusable components

4.1.1 Single Responsibility Principle - SRP

A module should have one, and only one, reason to change. It means that a module should be responsible to **one**, and only one, **actor**.

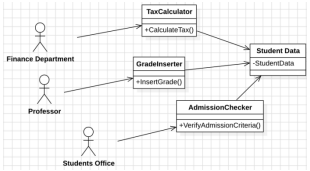
This is helpful to separate code used by different actors and for a better maintenance.

Example *CalculateTax()* is specified by the Finance Department. *InsertGrade()* is specified by Students Office and used by Professor *VerifyAdmissionCriteria()* is specified and used by Students Office *CalculateTax()* and *VerifyAdmission()* share a common algorithm for calculating the average grade.

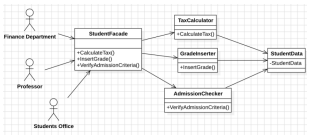


Changes in *ComputeAverage()* affect two actors: errors may arise and also changes to Class Student due to different needs of actors may lead to collisions and merges.

Solution 1: Separate data from functions



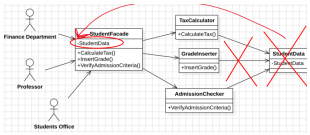
Solution 2: Use the Façade pattern



You increase cupling and complexity, its a trade off, but the separation is splitted.

> **Note:** You increase cupling and complexity but the separation is splitted.

Solution 2: variation



4.2 Open-Closed Principle - OCP

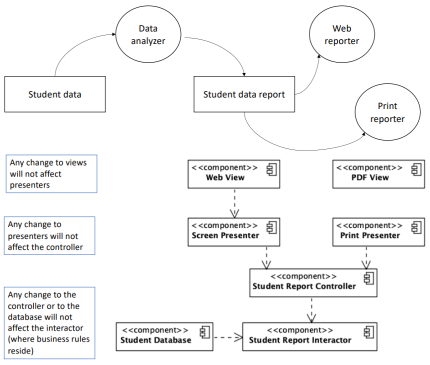
A software artifact should be **open for extension but closed for modification**. Using design patterns to make new funcitonality in a way that is addictive and not to modify the existing structure.

The behavior of a software artifact ought to be extendible, without having to modify that artifact.

The aim of OCP is to make the system easy to extend without requiring too many changes to the system.

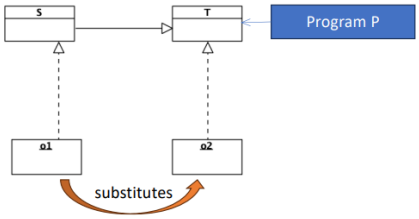
Using the open-closed principle, the software architect divides the system into components and arranges these components in a dependency hierarchy that protects higher-level components from changes in lower-level components.

Example A system displays summary information about a student's career at the university on a dashboard: Subjects (those passed with grades), Taxes paid and due, Current semester schedule, Next exams, Important notices, Stakeholders ask for a printed report with only information on subjects and taxes, and a different formatting.



Generating the report involves two separate responsibilities:
1. the calculation of the reported data
2 .the presentation of that data into a web- and printer-friendly form.

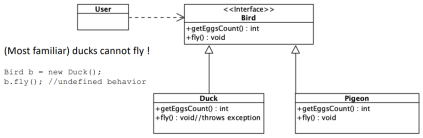
4.2.1 Liskov substitution principle - LSP



If for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o1 is substituted for o2 then **S is a subtype of T**.

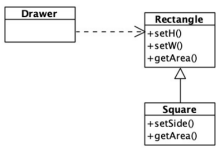
If we substrute o2 vitw o1 and the behavior of p does not change means S is derived class of T.

This principle is about subtyping, not only for inheritance or extension but also for the logic, the behavior of the class.
Derived type must fully support the substitution of their base types: functions that use pointers or references to base classes must be able to use objects of the derived without knowing it. This is related to the substitution property.
A subtype can replace its supertype only if it behaves the same way. This idea, called **behavioral subtyping**, means the subtype doesn't just have the same methods as the supertype — it also follows the same rules for how those methods should work. That way, anything the client expects from the supertype will also hold true for the subtype.




```
Rectangle r = ...;

r.setW(5); r.setH(2);
assert(r.area() == 10);
```

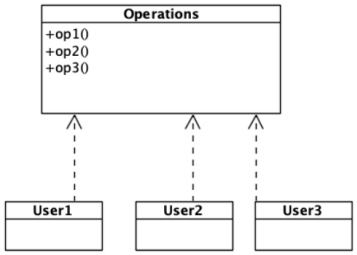


Depending on how r is defined, the assertion can fail because you not use the *setH()* and *setW()* methods of the rectangle but you use the *setSide()* method of the square.

How to refactor it? Using the Interface segregation principle - ISP

4.2.2 Interface segregation principle - ISP

Clients should not be forced to depend on operations and/or interfaces they do not use. Better to make fine grained interfaces that are client specific.
Similart to SRP we have different actors, represented by different classes. They are only interested in a subset of the operations, so we can split the interface in U1 use only operation1 and U2 use only operation2 and U3 use only operation3



In a statically typed language (e.g., Java), any change to *op1()* will force recompilation and redeployment of User2 and User3.

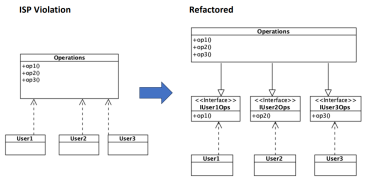
Is different from the SRP because we need the classes but we need to modularize better.

Inserting interfaces the classes use the interfaces instead the class and the interface implement the class.

If we change operation1 we impact also user2 and user3, if we change the interface we impact only the corrisponded user.

We also package the interface and the user toghether and this not impact the module.

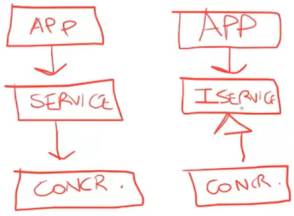
Operations are segregated into interfaces.
In a statically typed language (e.g., Java), a change to *op1()* will NOT force recompilation and redeployment of User2 and User3.



Remove useless operations from modules, otherwise they will force useless redeployments

4.2.3 Dependency inversion principle - DIP

You should always have dependencies that lower modules depend on higher modules, not the opposite.
High-level modules should not depend on anything from low-level modules.
Stable abstractions: abstractions should not depend on details, so that changes to details will not affect them.
Idealistic tension, in practice each system has at least a concrete component: the main function
We can make an interface:



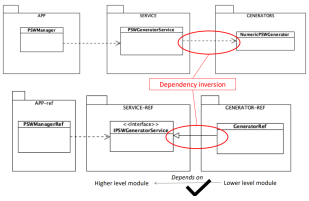
Example In Java (or other statically typed languages), this means that use, import and include statements should refer only to source modules containing interfaces, abstract classes or any other type of abstraction.

In dynamically typed languages, source code dependencies should not refer to modules in which functions being called are implemented.

Also we can package the app and the interface together.

4.2.4 DIP: practical implications

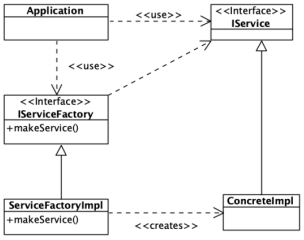
Dependencies on concretions are tolerable when it comes to operating systems, platform facilities and other similar components whose changes are tightly controlled and not frequent.
Abstract Factories are helpful to adhere to the DIP by minimizing the number of dependencies on concrete implementations and keep them separate from the rest of the system
In general, volatility should be shifted towards implementation details without changing interfaces.



- Don't refer to volatile concrete classes
- Don't refer from volatile concrete classes
- Don't override concreted functions
- Never mentions the name of anything concrete and volatile

Using Factory Pattern

To create an instance of ConcreteImpl, the Application calls *makeService()* from IServiceFactory interface, which is implemented by ServiceFactory-Impl class.
ConcreteImpl is created and returned as a IService.



Separation abstract and concrete levels: All source code dependencies point toward the abstract level. The flow of control crosses the separation line in the opposite direction: this is the sense of "dependency inversion".

DIP violation DIP violations cannot be entirely removed. Some dependencies on concrete implementations are necessary. They must be minimized and kept separate

4.3 How to compile and see the UML

```
mvn -f .\Path\to\pom_file\pom.xml clean compile site
```

Then Java Project > Target > site > index.html

Capitolo 5

Components

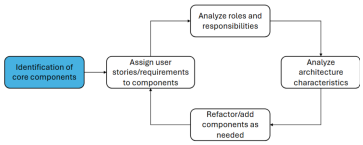
The logical components are the building blocks of the system. They usually manifest through a namespace or a directory structure, where leaf nodes containing source code represent the logical components.

Logical architecture: the system's logical components and how they interact each other.

Physical architecture: includes also physical artifacts, defining architectural

5.1 Identify components

Example: SPG = Solidarity Purchasing Group, Solidarity towards producers valuing (small vs large, local and seasonal vs global and specific vs mass).
Solidarity among clients: Not a normal shop but a group of people buying together, sharing the principles to select producers.
This is a software application for **Clients**, **Farmers** and **Shop employees**.



The identification of components is fundamental activity, necessary either for drafting architecture from scratch or reverse-engineer it. It is an iterative refinement process and three approaches can be used for initial identification:

- **Best guess**: based on the system core functionalities
- **Workflow**: Based on the "happy path"
- **Actor/action**: Based on major actions a user can perform

SPG requirements

A neighborhood association manages a Solidarity Purchasing Group (SPG). Every Monday, a farm (rotating among a pool of reference suppliers) submits to the SPG a list of available products, their quantities, and their prices.

Once this list is provided, the data is reviewed and confirmed by the SPG employee.

SPG members receive an email with the list of available products and can place their orders either via the website or by visiting the association's headquarters.

When ordering, SPG members specify the types of vegetables they want and the quantities (expressed in kilos, packs, or boxes depending on the type of fruit or vegetable ordered). If the order is placed at the association's headquarters, the member pays immediately.

At the end of the week, the farm delivers the products to the association's headquarters. Shop employees record the delivery, and members receive a notification when their ordered goods arrive. At this point, members can go to the association to collect the package (paying at the same time, if not done at the time of ordering).

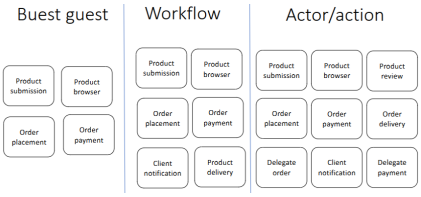
Macro functionality	**Component**
Insert products	Product submission
View products	Product browser
Order products	Order placement
Buy products	Order Payment

Workflow approach

Step	**Component**
1. Farmer submits available products, employee reviews	Product submission
2. SPG members/clients receive list of products	Client notification
3. Client browse the catalog of product	Product browser
4. Client places an order	Order placement
4a. Employee places an order for the client	Order placement
5. Client pays the order online	Order Payment
5a. Employee submit order payment	Order Payment
6. Employees register delivery of goods	Product delivery
7. Clients are updated on goods' delivery	Client notification
8. Clients retrieve goods	Product delivery

Actor/action

Actor	**Action**	**Component**
Client	Search for products	Product browser
	View details about products	Product browser
	Place order	Order placement
	Pay order	Order payment
Farmer	Insert products availabilities	Product submission
Shop employee	Review products availabilities	Product review
	Insert client order	Delegate order
	Insert client payment	Delegate payment
	Register product delivery	Order delivery
System !	Send email notifications	Email Notification



5.1.1 Putting components together

- **Tactic level - design**: Components principles: cohesion, coupling
- **Strategic level - architecture**: Component topology, technical partitioning vs domain partitioning, and Physical architecture, Monolithic vs Distributed architecture.
- **And more**: Deployment, communication style and data topology

5.2 Component principles

5.2.1 Component cohesion

How to put classes inside a component?

Cohesion is the degree to which the classes within a component belong together.

High cohesion means that the classes within a component are closely related in terms of functionality and purpose, while **low cohesion** indicates that the classes are more unrelated and serve different purposes.

It is suggested to follow principle:

- The **Reuse/Release Equivalence Principle - REP**: The granule of reuse is the granule of release.
- The **Common Closure Principle - CCP**: Classes that change together are packaged together.
- The **Common Reuse Principle - CRP**: Classes that are used together are packaged together.

Reuse/Release Equivalence Principle - REP

To support reuse, classes and modules that make up a component must: belong to a coherent group and be released together.

The release process shall produce appropriate notifications and release documentation, so that users can decide whether to integrate new releases.

Common Closure Principle - CCP

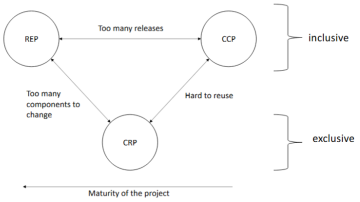
Group into components classes that change together, i.e. for the same reason and at the same time.

Separate into different components classes that have different changes processes and motivation.

> **Note:** There is a relationship between the Common Closure Principle (CCP). It is similar to SRP and it addresses the "closure" aspect of OCP

Common Reuse Principle - CRP

It help deciding which classes/module to place into the same components. In such components, usually classes have many dependencies on each other
Complementary perspective:



- users of a component should not depend on unnecessary features;
- if a class isn't closely connected to another class, it shouldn't be in the same component.

Similar to interface segregation design principle (ISP).

> **Note:** Similar to interface segregation design principle (ISP)

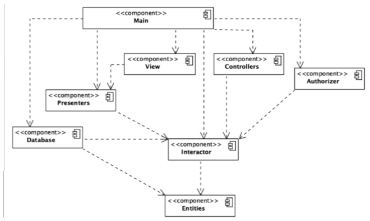
5.2.2 Component coupling

- The Acyclic Dependencies Principle (ADP): The dependency graph of packages must have no cycles.
- The Stable Dependencies Principle (SDP): Depend in the direction of stability.
- The Stable Abstractions Principle (SAP): Abstractness increases with stability.

Acyclic Dependencies Principle - ADP

Historical roots on typical issues of team-based development: the *morning after syndrome*[1] and the *weekly build*.

Solution to the problem: partition the development environment into releasable components
Each component has its own responsible developer/team so the releases are uniquely identified and components follow their own, private, development path. E.g. in git versioning system, using separated branches or even separated repositories.



- **Advantages**: Changes made to one component do not affect other teams, until they decide to adapt to the new component's release. I ntegration happens in small steps
- **Implications**: teams must monitor, manage and, maintain the dependency structure over time. there can be no "cycles" or circular dependencies in the dependency graph.

Example: directed acyclic graph Starting at any component, it is not possible to get back to the origin by following the edges (dependencies).

What happens when a change is made to Presenters? View and Main will be affected and tests with Interactor and Entities.

The bottom-up "building" for system release

- Entities
- Database and Interactor
- Presenters, View, Controllers, Authorizer
- Main

[1]The morning after syndrome occurs when code worked on, tested, and left working one day is broken the next morning due to changes made by others

Example 2: dependency cycle Now imagine a new dependency arises: User class in Entities uses the Permissions class in Authorizer.

Testing Entities requires now also Authorizer and Interactor.

There are different ways to solve this problem:

- Solution A: Use dependency inversion principle (DIP)
- Solution B: Create a new component on which both Entities and Authorizer depend. Move the class(es) they both depend on into this new component.

5.2.3 The Stable Dependencies Principle - SDP

Depend in the direction of stability. Stability is not about the frequency of changes, but about the amount of work required to make a change.

A cupboard is stable because it takes a lot of effort to move it, while a cup is not stable because it takes little effort to move it.

A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

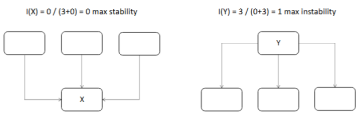
Modules that are designed to be harder to modify should not depend on modules that require little effort to change.

A simple measure of (in)stability

X is very stable because three components depend on it and it depends on no other component, while Y is very unstable because no component depends on it and it depends on three other components.

Instability: goes from 0 to 1, where 0 is the most stable and 1 is the most unstable.

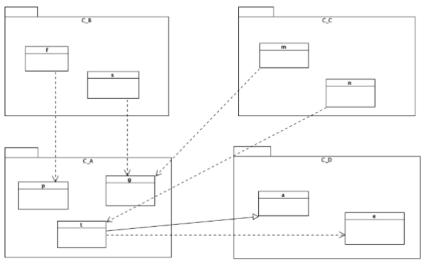
$$I = \frac{\text{Fan-out}}{\text{Fan-in} + \text{Fan-out}}$$



Where

- Fan-in is the number of incoming dependencies
- Fan-out is the number of outgoing dependencies

Example: I(C_B) = 0.33. Fan-in = 4 and Fan-out = 2. I(C_D) = $\frac{2}{4+2} = 0.33$.

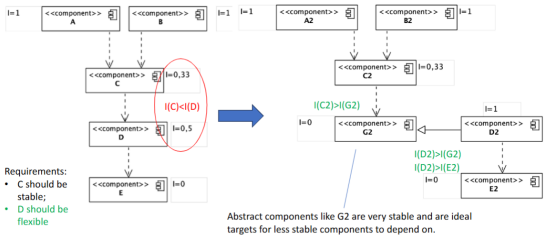


SDP and stability measure

Depend in the direction of stability.

The I(component) should be larger than the I(the components that it depends on). I metric should decrease in the direction of dependency. $I(C_B) = I(C_C) > I(C_A) > I(C_D)$

SDP violation as system evolves: example Requirements: C should be stable. D should be flexible.



5.2.4 Stable Abstractions Principle - SAP

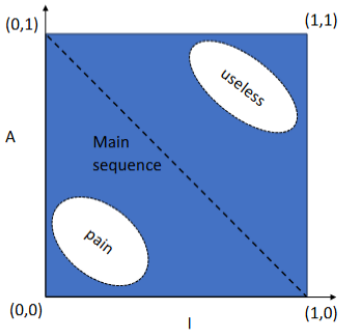
A component should be as abstract as it is stable. Code that represent high-level policies of the system should be placed into stable components.

SAP, OCP, SDP and abstract classes

Open Closed principle suggests creating classes that are flexible enough to be extended without requiring modification. Abstract classes are best suited to this need. Thus, applying SDP: stability goes into the direction of abstraction

A simple measure of abstractness this ratio:

$$A = \frac{\text{Number of abstract classes and interfaces in the component}}{\text{Total number of classes in the component}}$$



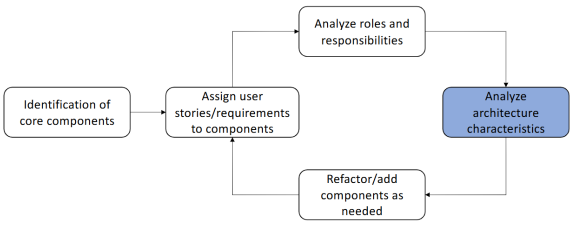
where:

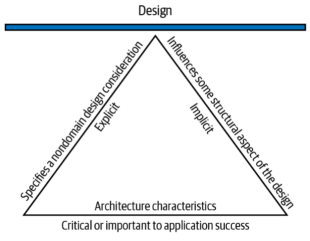
- A = 0 means that the component is completely concrete. No abstract classes.
- A = 1 means that the component is completely abstract. Only abstract classes.

Note: Distance from the main sequence: $D = |A + I - 1|$

5.3 Strategic decisions

5.3.1 Iterative Refinement





Operational Architectural Characteristics: examples

- **Scalability**: the system's ability to perform and operate as the number of users or requests increases
- **Elasticity**: the system's ability to perform and operate with peaks of requests
- **Responsiveness**: the system's ability to complete assigned tasks within a given time
- **Reliability/safety**: Whether the system needs to be fail-safe, or if it is mission critical in a way that affects lives. Fault tolerance does the software operate as intended despite hardware or software faults.

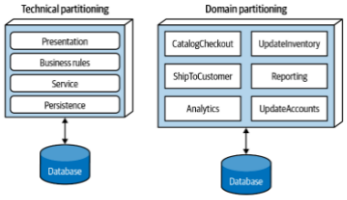
- **Availability**: How much of the time the system will need to be available; if that's 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure.
- **Continuity**: The system's disaster recovery capability.
- **Robustness**: The system's ability to handle error and boundary conditions while running, for example, if the internet connection or power fails.
- ...

Engineering Architectural Characteristics

- **Maintainability**: the degree of effectiveness and efficiency to which developers can modify the software to improve it, correct it, or adapt it to changes in environment and/or requirements.
- **Testablity**: how easily developers and others can test the software? How complete are tests?
- **Deployability**: ease, frequency and overall risk of deployment
- **Evolvability**: system's ability to survive changes in its environment, requirements and implementation technologies
- ...

5.3.2 Strategic level: partitioning

Technical partitioning vs domain partitioning



5.3.3 Conway's law

Organizations which design systems...are constrained to produce designs which are copies of the communication structures of these organizations.

The structure of the design will reflect how people communicate and work together

5.3.4 Monolithic versus Distributed architecture

Monolithic: single deployment unit of all code

Distributed: multiple deployment units connected through remote access protocols

Monolithic	Distributed
Layered architecture	Service-based
Pipeline architecture	Event-driven
Modular monolith	Space-based
Microkernel	Service-oriented
	Microservices

Capitolo 6

Architecture visualization: C4 notation

6.1 Visualizing software architecture

There are not only one diagrams to visualize software architecture, but many of them.
Variety of goals, stakeholders, focus, etc.

6.2 Issues in sw architecture visualization

- Purpose of elements (boxes, arrows, etc.)
- Relationship between elements
- Levels of abstraction
- No context or logical starting point
- Consistency between diagrams
- Require explanations or descriptions to be fully understood

Diagrams are like maps: different providers, google or openstreetmap have different purposes. For this reason, we will introduce the C4 model, which is a standard for software architecture visualization.

6.3 C4 Model

With this model we can zoom-in and zoom-out to navigate through the different levels of abstraction to use with different stakeholders.

A **software system** is made up of one or more **containers**, each of which contains one or more **components**, which in turn are implemented by one or more code elements. And **people** use the software systems that we build.

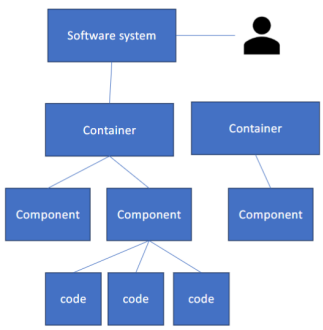


Figura 6.1: Static structure

It is organized in 4 levels of abstraction:

- Software System
- Container
- Component
- Code Element

6.3.1 Container

A boundary inside which some code is executed or some data is stored. Containers are **separately deployable**.

Examples:

- Application (server-side or client side)
- Database, file system, etc.
- Microservice (if not a system per se)
- Mobile app
- Script (python, shell, R, etc.)

6.3.2 Component

A grouping of related functionality encapsulated behind a well-defined interface

Components are **NOT** separately deployable and how components are packaged is not relevant at this level.

6.3.3 Code

The basic building blocks of a programming language:

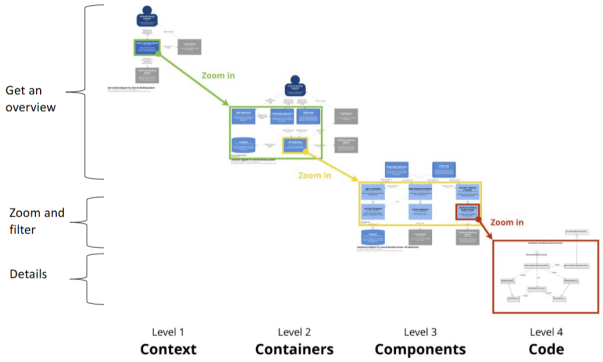
- classes,
- interfaces,
- functions,
- objects,
- enums

6.4 Different type of diagrams

The C4 model defines different types of diagrams for each level of abstraction.

- **Context diagram**: it shows how the software system in scope fits into the context around it.
- **Container diagram**: it shows the high-level technical building blocks (containers) and how they interact.
- **Component diagram**: it shows the components inside each container
- **Code diagram**: it shows how a component is implemented

Using the SPG, Solidarity Purchasing Group, presented before, as an example, we can create different diagrams for each level of abstraction.



6.5 Level 1: Context diagram

It helps answering such questions:

- What is the software system that we are building (or have built)?
- Who is using it?
- How does it fit in with the existing environment?

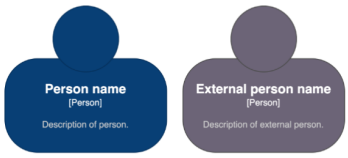
It is **required**: it is the starting point for communicating with any stakeholder
The key elements of a context diagram are:

- People who will use the software system
- Other software systems with which the reference system interacts
- + optionally the organization boundary

Interaction between elements: Directed arrows with high-level information about the purpose of that interaction. We can connect elements with arrows. It is better to use only one way arrows and not to use bidirectional once.

6.5.1 Information reported - People

The people element have:



- **Name**: the name of the person, user, role, actor or persona.
- **Description**: A short description of the person, their role, responsibilities, etc

6.5.2 Information reported - Software system

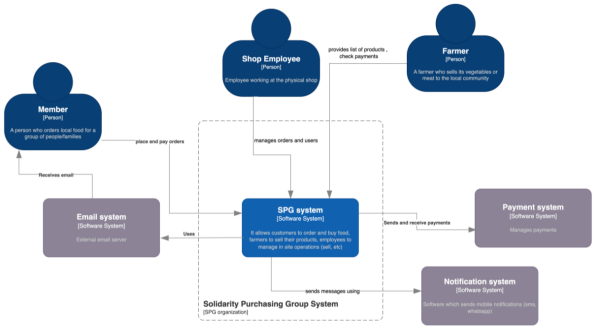
The software system element have:

- **Name**: The name of the software system.
- **Description**: A short description of the software system, its goal and responsibilities, etc.

6.6 Level 2: Container diagram

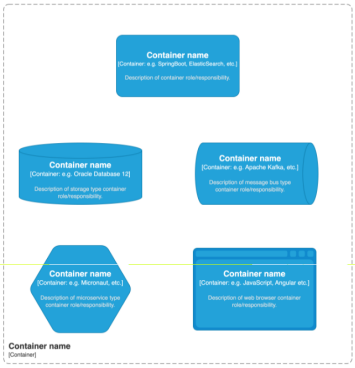
Goal: illustrate the high-level technology elements and how responsibilities are distributed across elements and how containers communicate with each other

Every container must be deployable independently and the diagram is **required** and readed by software development adn operations stuff.



6.6.1 Elements

People, software systems and containers. For each container:

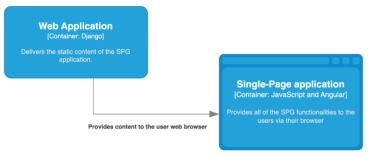


- Name: The name of the container (e.g. “Internet-facing web server”, “Database”, etc).
- Technology: The implementation technology (e.g. Java/Spring MVC application, ASP.NET web application, Relational Database Schema, etc.) or the general type of technology”
- Description: A short descriptive statement, or perhaps a list of the container’s key responsibilities or entities/tables/files/etc. that are being stored.

System boundary must be coherent with L1

6.6.2 Interactions between containers

Interactions between containers must be as explicit as possible. Example of useful information to annotate the interactions with:



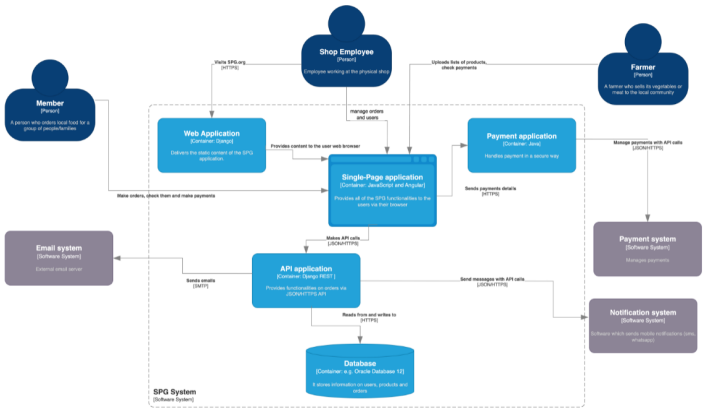
- The purpose of the interaction (e.g. “reads/writes data from”, “sends reports to”, etc).
- The communication mechanism (e.g. REST, Web API, Java Message Service, Web Services).
- The communication style (e.g. synchronous, asynchronous, batched, twophase commit, etc).
- Protocols and port numbers (e.g. HTTP, HTTPS, SOAP/HTTP, SMTP, FTP, RMI/IIOP, etc)

6.7 Level 3: Components diagram

It shows components that are within a single container at a time. Infrastructural elements (e.g. , logging, metering) and shared library might be annotated, separated, or excluded.

It helps to answer the questions like

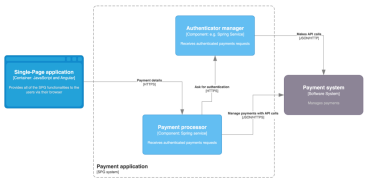
- What components is each container made up of?
- Do all components stay within in a container?
- How components interact each other ?
- How the software works at a high-level?



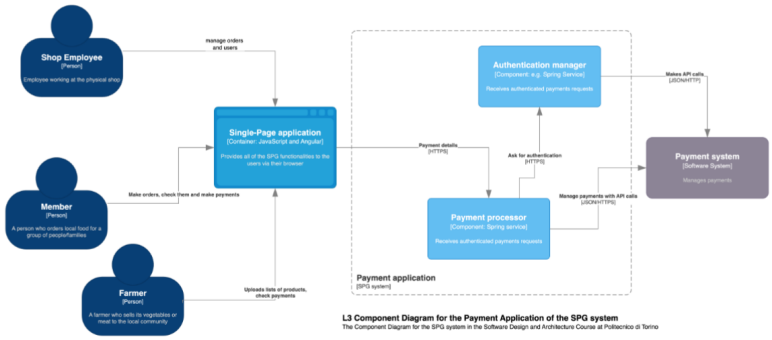
Optional, especially for containers like data storage, microservices. It is for technical people within the software development team.

6.7.1 Elements

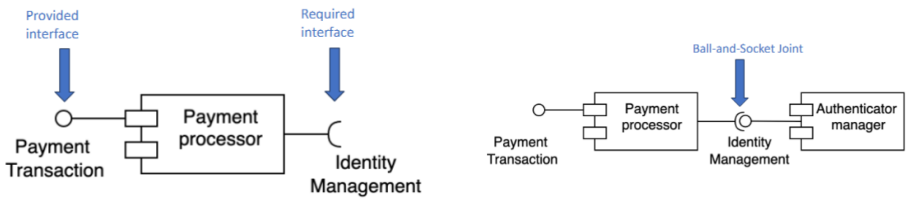
Same of the containers + components: people, software systems, containers and components.
Information to show for each component:



- Name: The name of the component.
- Technology: The implementation technology for the component (e.g. Java, Object, C#, Ruby).
- Description: A short description of the component in terms of its responsibilities

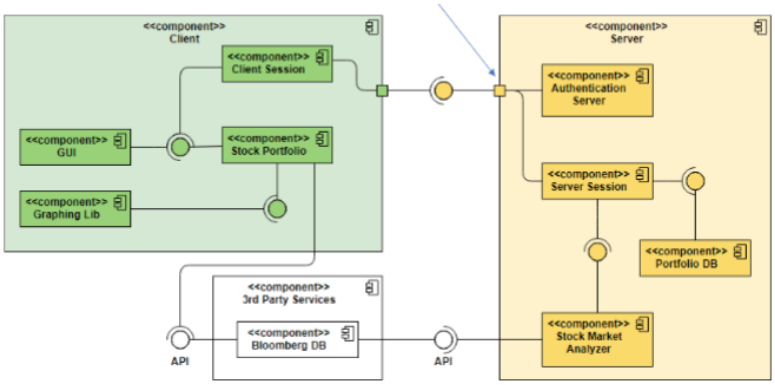


The UML component diagram is useful to provide more detailed documentation.



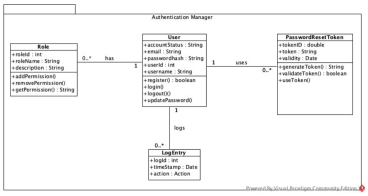
Ports

Ports indicate that required/provided interfaces are delegated to an internal class.



6.8 Level 4: Code diagram

It shows the implementation details of an individual component, it helps bridging the gap between architecture and code.
Usually, UML class diagrams are used beacuse:



- They can be generated automatically (e.g., with PlantUML)
- Large amount of details !

This is not mandatory, it is optional and it is especially for software developers, and more in general for technical people within the software development team.

We can use Draw.io but no check on syntax. Instead we can use `https://academic.signavio.com/p/login`.

Coerence between levels are important for exam.

6.9 Summary of C4 model

A lightweight notation to visualize the static structure of a software system It is based on a hierarchic conceptual model of software systems. 4 levels (2 of which mandatory).

Four main elements, associated with unidirectional lines, directions shown is the most relevant.

Customizable with colors , shapes, lines

- line styles for synchronous/asynchronous communication;
- colors for different components;
- different shapes for specific types of components;